Computer Vision and Cognitive Science, Part 2: Deep Networks and Bayes Decision Theory

Lecture 15

Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

20 October, 2026

What this lecture covers

Features and deep networks

Transformers, foundation models

Generative models, conjectures

Summary

Colors used in the figures

These notes follow the second half of an article by Yuille and Kersten (2025), abbreviated YK, on computer vision and cognitive science from the perspective of Bayes decision theory; Lecture 2 covered the first half.

In the formulas and the diagrams the colors keep one meaning.

- Blue marks the Bayesian route: the likelihood $P(I \mid W)$, the prior $P(W)$, analysis by synthesis, and the input image I.
- Teal marks the direct route: $P(W \mid I)$ learned from data, the decision rule $\alpha(I)$ and the network computing it, and correct labels.
- Orange marks data and annotation, and a network's backward pass.
- Purple marks networks and their layers.
- Olive marks hand-designed features and wrong labels.

Some figures also use these and other colors only to tell elements apart (the ventral stream's stages, a backbone's heads, the true image-caption pairs); the captions and keys say what they mark. A caption citing "Lecture N, slide M" took its picture from that slide (Yuille, 2026).

Recap: Bayes decision theory and two ways to use data

Vision is an inverse problem: from an image I, infer the state of the world W that produced it, such as the objects, their shapes and positions, and the lighting. The likelihood $P(I \mid W)$ says how images are generated from a world state; when a graphics engine specifies it, vision becomes inverse computer graphics. The prior $P(W)$ says which world states are likely. Bayes' rule combines them into the posterior

$$P(W \mid I) = \frac{P(I \mid W)\, P(W)}{P(I)},$$

the probability of each world state once the image has been seen, where $P(I) = \sum_W P(I \mid W) P(W)$. Many world states can produce the same image; the prior selects among them.

Bayes decision theory (BDT; Berger, 1985) adds two ingredients. A decision rule $\alpha(I)$ maps each image to a decision, for instance an estimate of W. A loss function $L(W, \alpha(I))$ says what the decision costs when the true state is W. The risk of a rule is its expected loss, $R(\alpha) = \sum_{I,W} L(W, \alpha(I))\, P(I, W)$, and BDT chooses the rule with the least risk.

The best decision depends only on the posterior

Write $P(I, W) = P(W \mid I)\, P(I)$; the risk becomes

$$R(\alpha) = \sum_I P(I) \sum_W L(W, \alpha(I))\, P(W \mid I).$$

The decision for one image leaves the terms of the other images unchanged, so the risk is smallest when, for every image, $\alpha(I)$ is the decision a with the smallest inner sum, the expected loss given that image:

$$\alpha^*(I) = \arg\min_a \sum_W L(W, a)\, P(W \mid I).$$

Only the posterior and the loss enter this formula.

If every error costs the same, $L(W, a) = 0$ when $a = W$ and 1 otherwise (the 0–1 loss), the inner sum equals $1 - P(W{=}a \mid I)$, and the best decision is the most probable world state, $\alpha^*(I) = \arg\max_W P(W \mid I)$, the maximum a posteriori (MAP) estimate.

Recap: Bayes decision theory and two ways to use data

BDT assumes that $P(I, W)$ is known. In practice we have data: n annotated examples (I_i, W_i), $i = 1, \ldots, n$, images together with their world states or labels, drawn from one source. YK distinguishes two ways to use them.

The Bayesian route learns the likelihood $P(I \mid W)$ and the prior $P(W)$ from the data, or specifies them (for instance by computer graphics), and computes the posterior by Bayes' rule.

The direct route learns the posterior $P(W \mid I)$, or the decision rule $\alpha(I)$ itself, from the data. A rule is chosen by minimizing the empirical risk, the average loss over the examples,

$$\hat{R}(\alpha) = \frac{1}{n} \sum_{i=1}^{n} L(W_i, \alpha(I_i)).$$

For a rule fixed in advance, and examples drawn from $P(I, W)$, the empirical risk approaches the risk as n grows (the law of large numbers). Neural networks are usually trained to learn $P(W \mid I)$, a special case of this route.

Recap: Bayes decision theory and two ways to use data

Which route should be taken? From the BDT perspective the best rule depends only on the posterior, so arguably there is no need to learn the likelihood and the prior.

A counterargument is that, when data is limited, the likelihood and the prior may be easier to model. The vision modules of Lecture 2 (stereo, shape from shading) were built this way, and so are many models of cognition (Griffiths et al., 2024). The factorization can also transfer: YK cites Bayesian edge detection (Konishi et al., 2003), where an algorithm trained on one type of data, images of house interiors, generalized and performed well on another, images of the countryside.

The direct route needs only what the decision needs, but it needs large annotated datasets, and it struggles on data from a source that differs significantly from its training data. The likelihood is the hard part of the Bayesian route: until recently, specifying $P(I \mid W)$ for real images was impractical, beyond simple vision modules.

Recap: Bayes decision theory and two ways to use data

In the figure the annotated data (orange) feed both routes. The Bayesian route learns $P(I \mid W)$ and $P(W)$, then applies Bayes' rule; the direct route learns $P(W \mid I)$ or $\alpha(I)$ directly. Both end in the decision with the least risk.

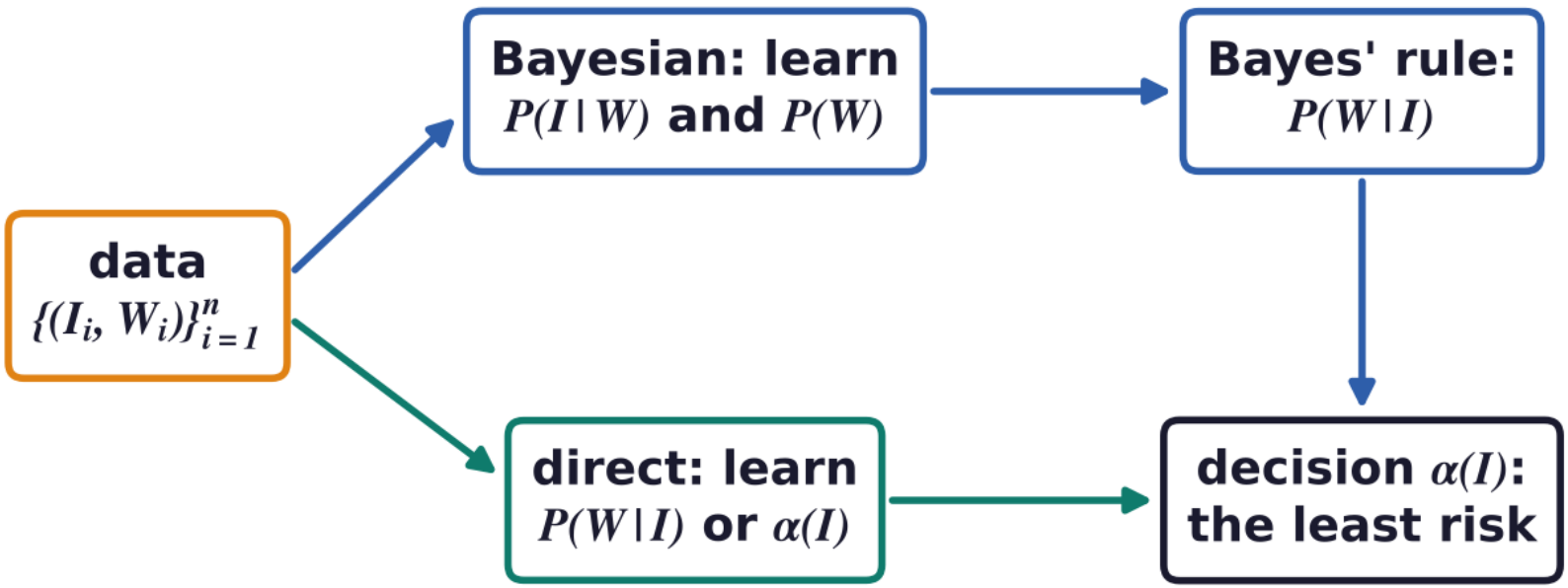


Figure 1: The two ways to use data (after Lecture 2, slide 35).

Features give a straightforward way to process images

Features have a long history in computer vision because they give a straightforward way to process images. A feature is a number computed from the image at each pixel, usually by a filter. Filtering, or convolution, replaces the intensity at each pixel x by a weighted sum of the intensities around it,

$$(k * I)(x) = \sum_u k(u)\, I(x - u),$$

where the kernel k holds the weights; the same kernel is used at every pixel.

The simplest features are derivatives. Kernels for $\partial/\partial x$ and $\partial/\partial y$, smoothed by a Gaussian, give the gradient $\nabla I(x) = (\partial I/\partial x, \partial I/\partial y)$, and its magnitude $|\nabla I(x)|$ is large where the intensity changes fast. Edge detection convolves the image with a derivative filter and thresholds the result: pixel x is marked as an edge if $|\nabla I(x)| > T$. Lecture 4 chose the threshold T by Bayes decision theory, from the distributions of the filter response on edges and on non-edges.

Features give a straightforward way to process images

The two small squares are the kernels, the y derivative (upper) and x derivative (lower) of a Gaussian of width $\sigma = 1.5$ pixels (gray 0, white positive, black negative); $|\nabla I|$ is bright where it is large. The threshold T, the 88th percentile of $|\nabla I|$, marks 12 % of the pixels, drawn black: the outlines of the eyes and nose, the whisker pads and the stripes of the fur.

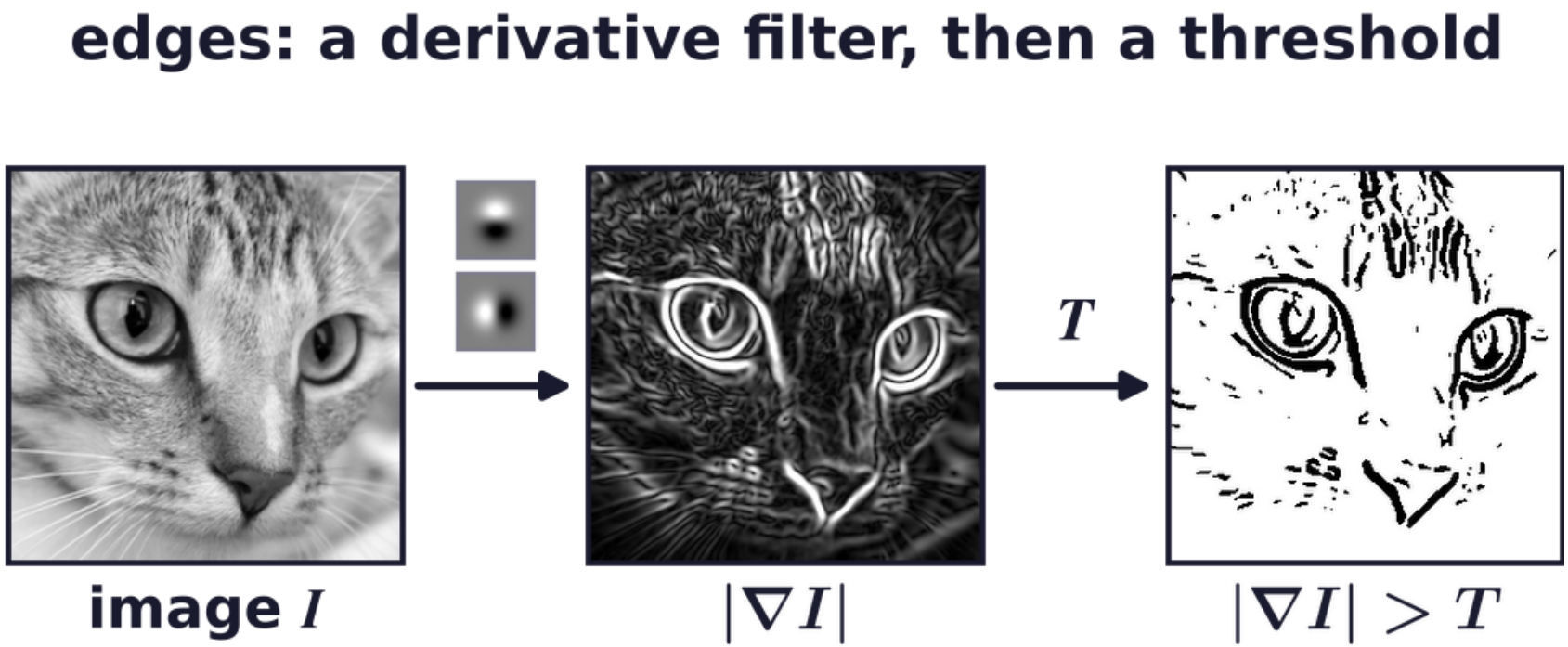


Figure 2: Edges of scikit-image's "chelsea" (public domain), in gray, 240 × 240 pixels: a derivative filter, then a threshold.

A Gabor filterbank describes each part of an image

A Gabor filter is a sinusoid of orientation θ and frequency f inside a Gaussian window of width σ,

$$g(x, y) = \exp\Big(- \frac{x^2 + y^2}{2\sigma^2} \Big) \cos(2\pi f x_\theta + \phi), \qquad x_\theta = x \cos\theta + y \sin\theta.$$

Its stripes run perpendicular to the direction θ, and it responds to stripes and edges of that orientation and spatial frequency near each pixel. The phases $\phi = 0$ and $\phi = \pi/2$ form a pair, and the square root of the sum of their squared responses measures the local energy at that orientation, whatever the exact position of the stripe. A filterbank applies several such filters, at several orientations and scales, and gives each pixel a vector of responses: a local description of the image around it. The statistics of these vectors over a region classify textures. Matching them between two images addresses the correspondence problem of binocular stereo and motion: finding the pixel of one image that shows the same point of the world as a given pixel of the other image. These filters are designed by hand; deep networks learn their filters from data.

A Gabor filterbank describes each part of an image

Upper row: four Gabor kernels (phase $\phi = 0$); under each, the energy of its pair at every pixel, on one brightness scale. Bright marks run along each kernel's stripes: vertical for the first, horizontal for the third.

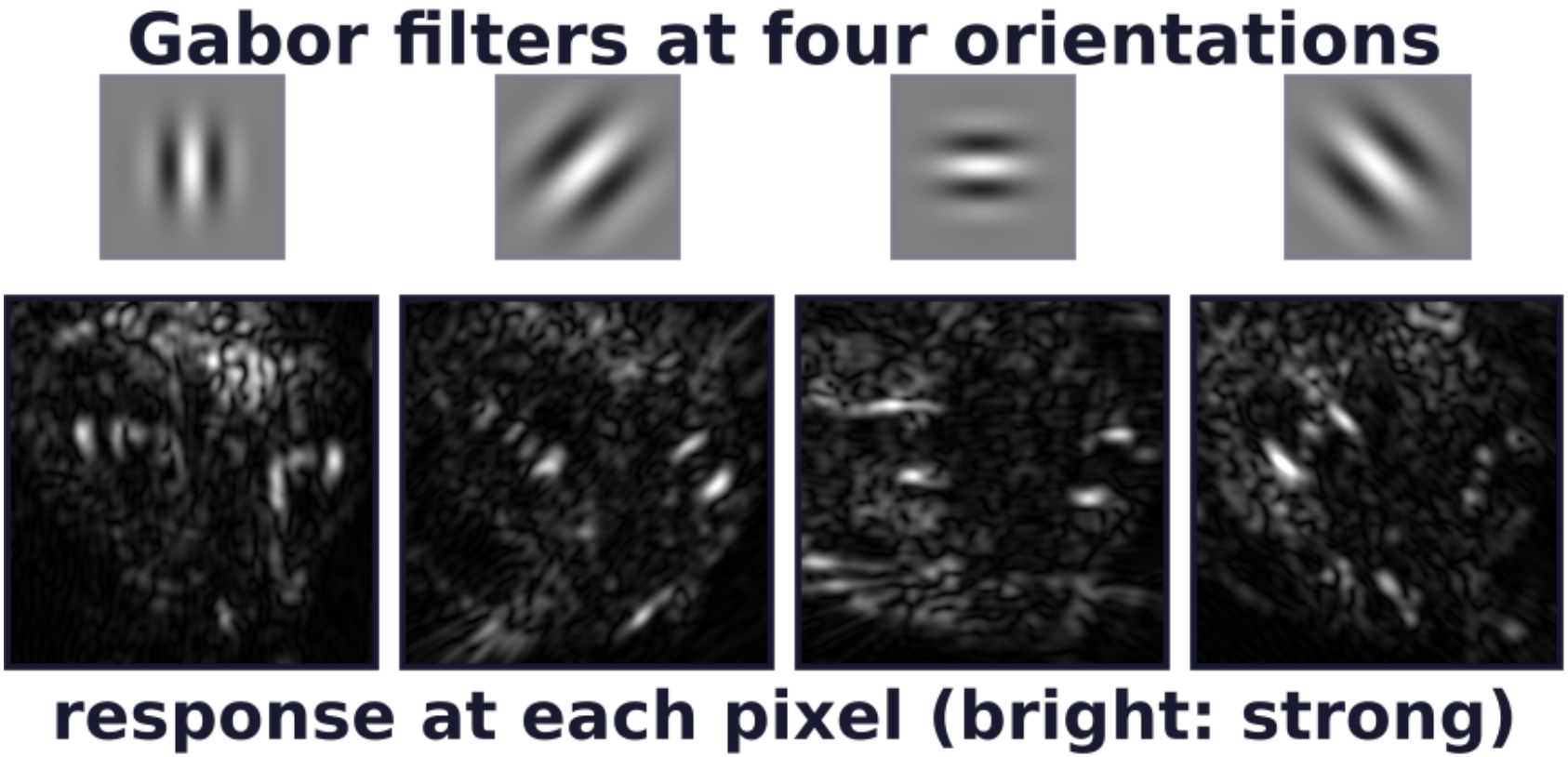


Figure 3: Gabor kernels at 0°, 45°, 90°, 135° (0.16 cycles per pixel, $\sigma = 3.5$ pixels) and their responses on the photograph of Figure 2.

The ventral stream inspired hierarchical models of cortex

The ventral stream is the pathway of visual areas from the primary visual cortex (V1) through the areas V2 and V4 to the inferotemporal cortex (IT), and it is associated with the recognition of objects. Each neuron has a receptive field: the region of the image where a stimulus can make it respond.

Studies of the mammalian ventral stream showed that it has a hierarchical structure. Low in the hierarchy, in V1, many neurons behave like filters similar to Gabor filters: small receptive fields, tuned to an orientation and a spatial frequency. Higher up the hierarchy, receptive fields are larger, and neurons are more specific to image patterns and less tuned to their position in the image: a neuron in IT may respond to a particular shape, or to a face, over a large part of the visual field.

The ventral stream inspired hierarchical models of cortex

This hierarchy inspired computational models of the visual cortex. The neocognitron (Fukushima, 1980) alternates two kinds of layers. S-cells (simple cells) extract features by matching local templates; C-cells (complex cells) combine the S-cells of nearby positions, so that their response tolerates small shifts. Stacking such pairs builds tolerance to position up the hierarchy. The input connections of the S-cells were learned without labels (self-organization); those of the C-cells are fixed.

A more realistic model (Riesenhuber and Poggio, 1999) starts from Gabor filters, alternating template matching with a maximum over nearby positions and scales; its top units, like IT cells, prefer object views.

A multi-layer perceptron is a feedforward neural network: the input passes through layers of units, each unit applying a nonlinearity to a weighted sum of all the units of the layer before (a fully connected layer), with weights learned by backpropagation (Lecture 10). Modified to use local receptive fields and the same weights at every position, it became the convolutional neural network (CNN; LeCun et al., 1989), trained on images of handwritten zip codes, a forerunner of modern deep networks.

The ventral stream inspired hierarchical models of cortex

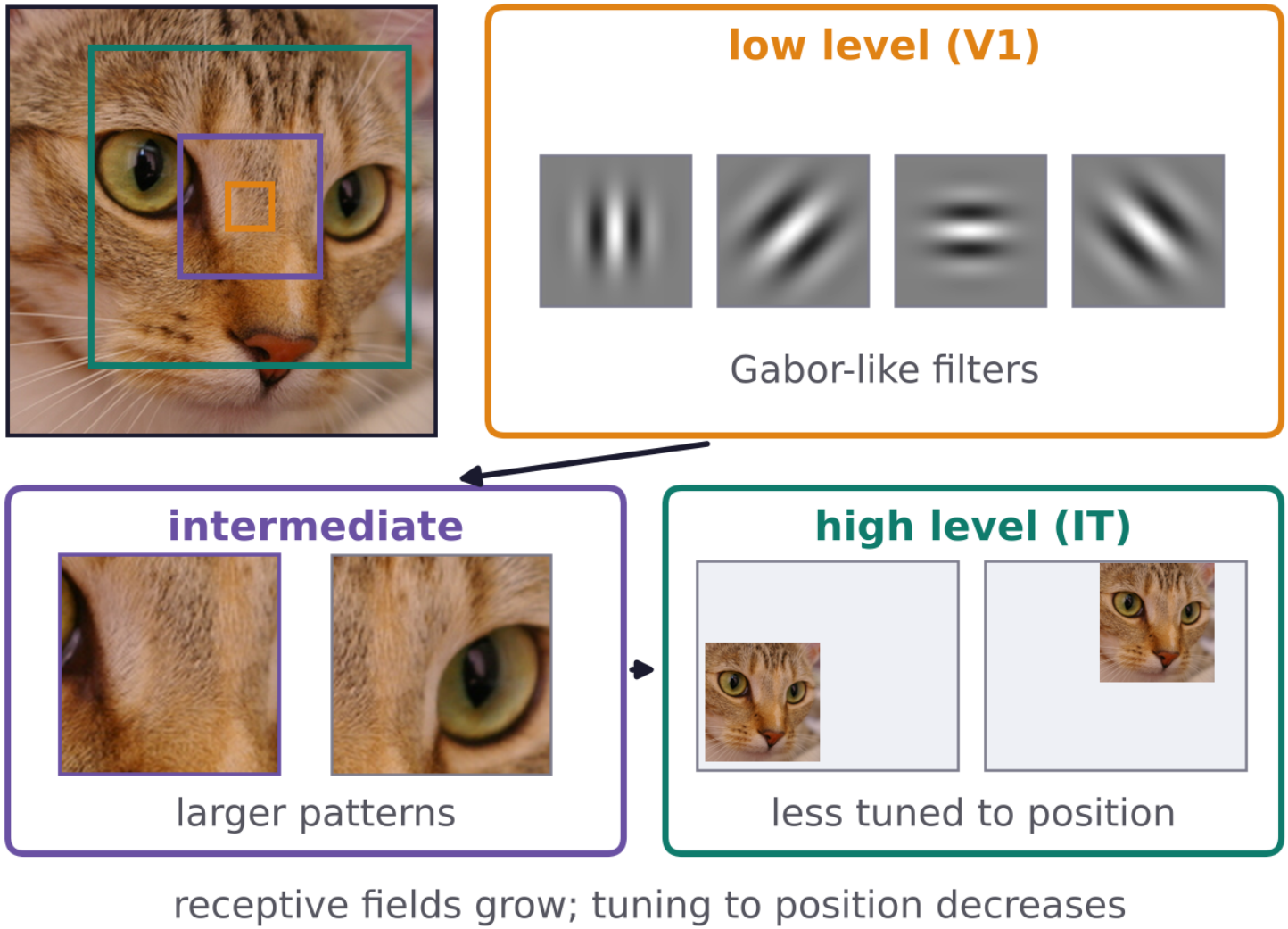


Figure 4: The ventral stream, schematically. Nested squares on the photograph: receptive fields of the three levels, in their stages' colors. Intermediate: the purple field and one of its size over the eye. High level: one face at two positions, treated alike.

A neural network specifies a class of decision rules

From the BDT perspective, a neural network with weights ω computes a decision rule $\alpha(I;\omega)$. Every setting of the weights gives one rule, so the network specifies a class of decision rules parameterized by ω, and learning chooses the rule of the class with the least empirical risk,

$$\hat{R}(\omega) = \frac{1}{n}\sum_{i=1}^{n} L(W_i, \alpha(I_i;\omega)).$$

To use calculus, the empirical risk must be a differentiable function of ω. The 0–1 loss is piecewise constant in ω: its gradient is zero almost everywhere. So the network usually outputs a posterior $P(W \mid I;\omega)$, for instance by a softmax, which turns one score per class into positive probabilities that sum to 1, and the loss is

$$L = -\log P(W_i \mid I_i;\omega).$$

Minimizing its average maximizes the probability given to the true labels, as for logistic regression (Lecture 10): the network learns the posterior, the direct route, and decides by its most probable state.

A neural network specifies a class of decision rules

The network computes the decision from the image I through the hidden layers H_1 and H_2, which lie between the input and the output (each circle is a unit); ω_1, ω_2, ω_3 are the weights of the three layers. The gray arrows are the forward pass, which computes $\alpha(I;\omega)$ and its loss against the ground truth W. The orange arrows are the backward pass, which carries the derivatives of the loss with respect to each layer back through the network; each $\partial L/\partial\omega_l$ is computed from them.

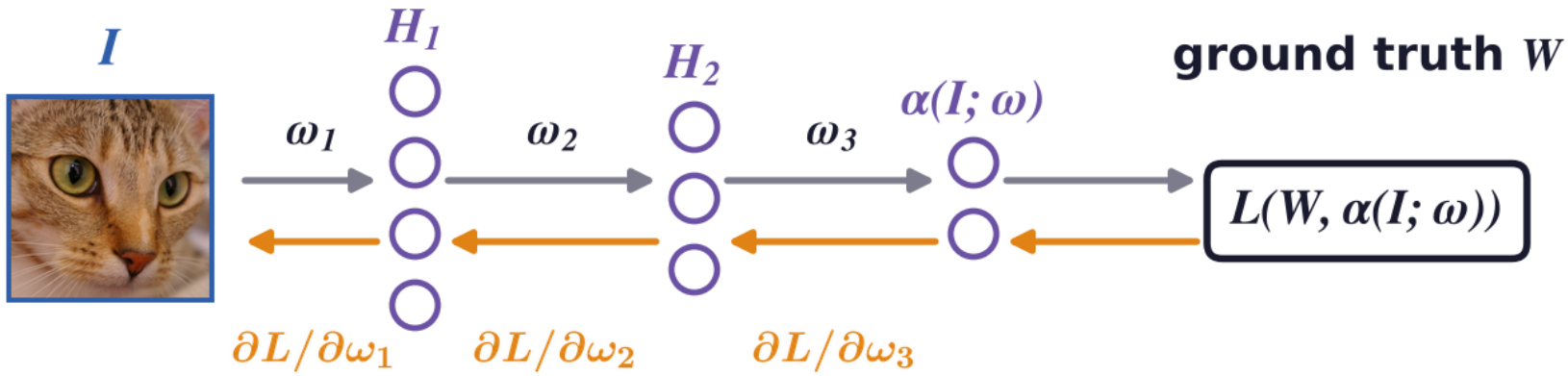


Figure 5: A three-layer network as a decision rule (after Lecture 10, slide 20).

Stochastic gradient descent and backpropagation

Gradient descent lowers $\hat{R}$ by small steps against its gradient, $\omega \leftarrow \omega - \eta\, \partial \hat{R}/\partial \omega$, with a small step size η; each step costs a pass over all n examples. Stochastic gradient descent (SGD) picks one example i at random and steps against the gradient of its loss alone,

$$\omega \leftarrow \omega - \eta \frac{\partial L(W_i, \alpha(I_i; \omega))}{\partial \omega}.$$

Averaged over the random choice of i, this gradient equals $\partial \hat{R}/\partial \omega$, so each step goes downhill on average, at a cost independent of n; in practice a minibatch of a few examples is averaged.

Backpropagation computes the gradient by the chain rule. With hidden layers H_1, H_2 and output α,

$$\frac{\partial L}{\partial \omega_3} = \frac{\partial L}{\partial \alpha}\frac{\partial \alpha}{\partial \omega_3}, \quad \frac{\partial L}{\partial \omega_2} = \frac{\partial L}{\partial \alpha}\frac{\partial \alpha}{\partial H_2}\frac{\partial H_2}{\partial \omega_2}, \quad \frac{\partial L}{\partial \omega_1} = \frac{\partial L}{\partial \alpha}\frac{\partial \alpha}{\partial H_2}\frac{\partial H_2}{\partial H_1}\frac{\partial H_1}{\partial \omega_1}.$$

The products share their first factors, computed once and passed down, so the backward pass costs only about twice the forward pass.

CNNs succeeded with large annotated datasets and GPUs

Computer vision researchers were skeptical of CNNs, because deformable part models (DPMs; Felzenszwalb et al., 2008) did better on the smaller datasets of the time. A DPM describes an object by a coarse template for the whole object and finer templates for its parts, whose positions may shift relative to the whole at a cost; the templates are trained to tell the object from the background, on hand-designed features computed from histograms of image gradients.

This changed with AlexNet (Krizhevsky et al., 2012), which won the 2012 ImageNet challenge on object classification. The ImageNet Large Scale Visual Recognition Challenge (ILSVRC; Russakovsky et al., 2015) labels each image with one of 1,000 classes, an entry names five classes per image, and the 2012 training set has 1.28 million annotated images. AlexNet could be trained on so much data partly because of its computational efficiency. Its convolutional layers share features: the same filters are applied at every position, so they need far fewer weights than fully connected layers. And it was trained on two graphics processing units (GPUs), processors designed for computer graphics that perform many multiplications in parallel.

CNNs succeeded with large annotated datasets and GPUs

Each bar is the top-5 error of the winning entry of the challenge in one year: the percentage of test images whose true class is not among the five classes the entry ranks most probable. Olive bars are entries with hand-designed features, teal bars are CNNs. The values are for entries trained on the challenge data only; AlexNet's winning entry used extra ImageNet data and reached 15.3 %, and Clarifai's 2013 entry with extra data 11.2 %. The error fell from 28.2 % in 2010 to 3.6 % in 2015.

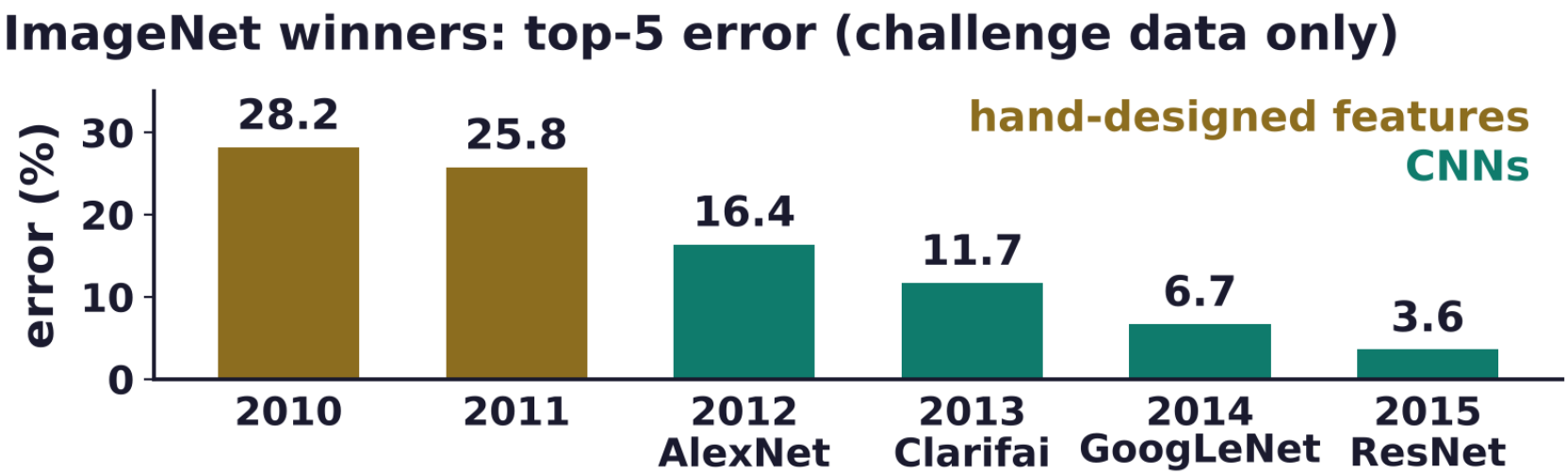


Figure 6: Top-5 error of the ILSVRC winners (Russakovsky et al., 2015, Tables 5 to 7; 2015: He et al., 2016).

Residual connections let very deep networks train

Beyond some depth, adding layers to a plain stack of layers increased even its training error (He et al., 2016). ResNet, the residual network, adds a residual connection around each block of layers,

$$H_{l+1} = H_l + F(H_l; \omega_l),$$

where H_l is the input of block l, and F, two weight layers with a rectified linear unit (ReLU, $\max(0, u)$) between them, computes a correction. The original ResNet applies one more ReLU after the addition; the formulas here, like the figure, leave it out. When the best block is close to the identity, F only has to learn a small residual. The identity path also carries the gradient back unchanged, $\partial H_{l+1}/\partial H_l = \mathrm{Id} + \partial F/\partial H_l$, where Id is the identity, so the derivative with respect to an early layer keeps a direct term from every later block and does not shrink to a product of many small factors. Networks of 152 layers trained well, and an ensemble of six residual networks, two of them 152 layers deep, won the 2015 challenge.

Residual connections let very deep networks train

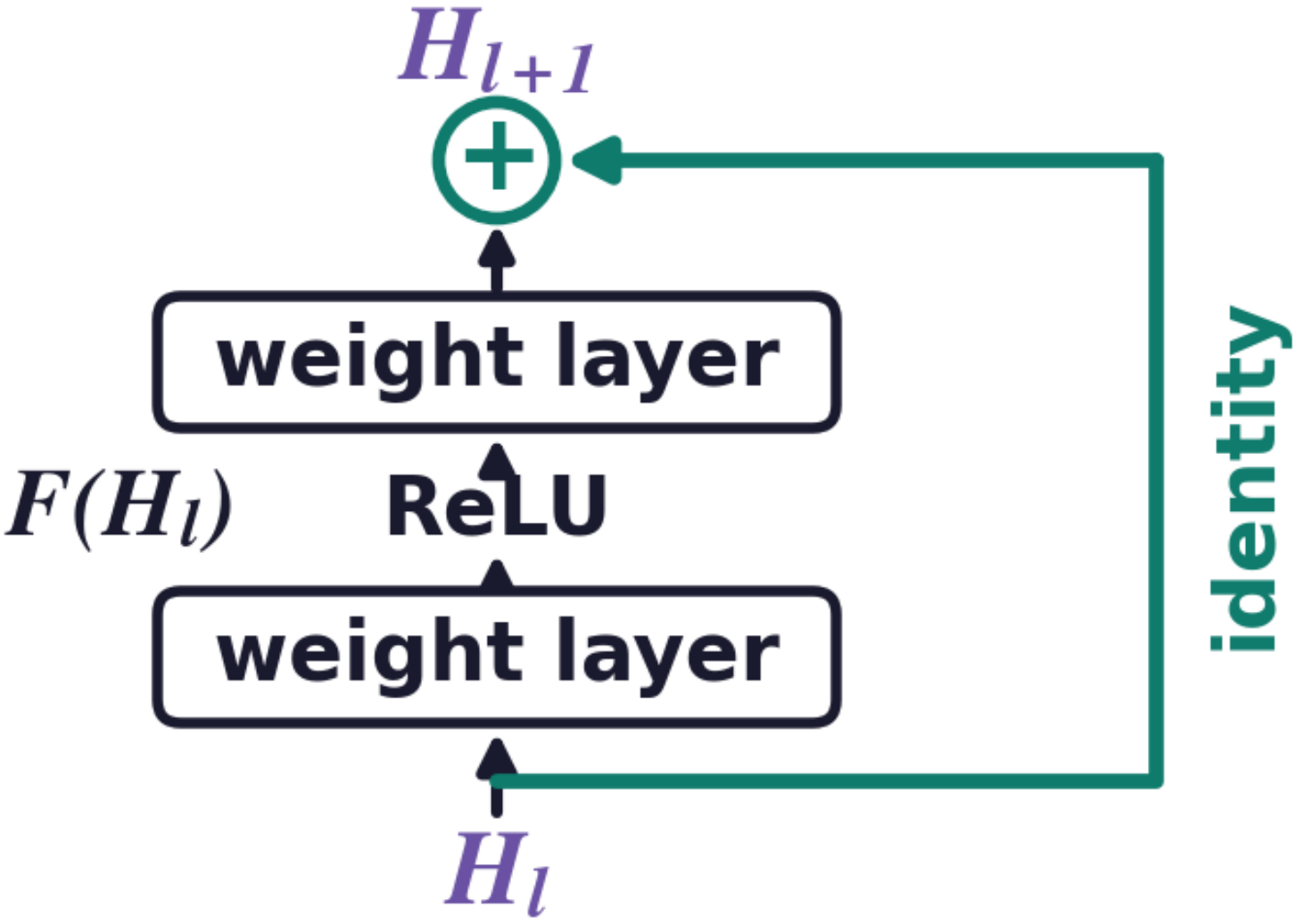


Figure 7: A residual block (Lecture 10, slide 41): two weight layers compute $F(H_l)$, and the identity path (teal) adds H_l to give H_{l+1}.

CNNs were rapidly applied to all tasks with annotated data

CNNs were rapidly applied to all visual tasks with large annotated datasets, with enormous increases in performance.

Semantic segmentation labels every pixel with a semantic class, such as person, cat or background. A fully convolutional network (FCN; Long et al., 2015) replaces the fully connected layers of a classification network by convolutions, so it outputs a coarse map of class scores (one number per class), upsampled to the size of the image; connections from earlier, finer layers restore detail. The figure's FCN-ResNet50 (a 50-layer ResNet; 20 object classes and background) upsamples by interpolation alone.
Edge detection: holistically-nested edge detection (HED; Xie and Tu, 2015) puts an edge output and a loss on every stage (block of layers) of a CNN (Lecture 10).

Two-dimensional pose estimation finds human joints, such as shoulders, elbows and wrists. Chen and Yuille (2014) combine a CNN with a graphical model of the body (Lectures 5 and 6): the CNN predicts each part and how neighboring parts are placed relative to each other, and these image-dependent relations enter the terms that score pairs of parts.

CNNs were rapidly applied to all tasks with annotated data

Pixels labeled as the object (a cat, then a person) are tinted teal inside a teal outline; the other pixels, almost all labeled background, are faded. A few pixels by the flag, at the left edge, are also labeled person.



Figure 8: Semantic segmentation (Lecture 10, slide 38) of "chelsea" and of scikit-image's "astronaut" (NASA, public domain).

New architectures tackled tasks once considered intractable

These successes inspired researchers to create datasets and to develop new network architectures for vision tasks that had been considered intractable.

Action recognition labels a video clip with the action it shows, such as a jump or a handshake. The two-stream network (Simonyan and Zisserman, 2014) combines image and motion cues. A spatial stream, a CNN, sees one frame, which shows the scene and the objects. A temporal stream, another CNN, sees the optical flow of several consecutive frames, stacked: the optical flow is the apparent motion of each pixel from one frame to the next, a two-dimensional vector at every pixel. Each stream outputs class scores, and the two are fused, either averaged or combined by a linear classifier (a support vector machine) trained on them.

New architectures tackled tasks once considered intractable

One frame (scikit-image's "astronaut") feeds the spatial CNN. A stack of optical-flow fields feeds the temporal CNN; the purple arrows stand for the motion of the pixels. The class scores of the two are fused into the action.

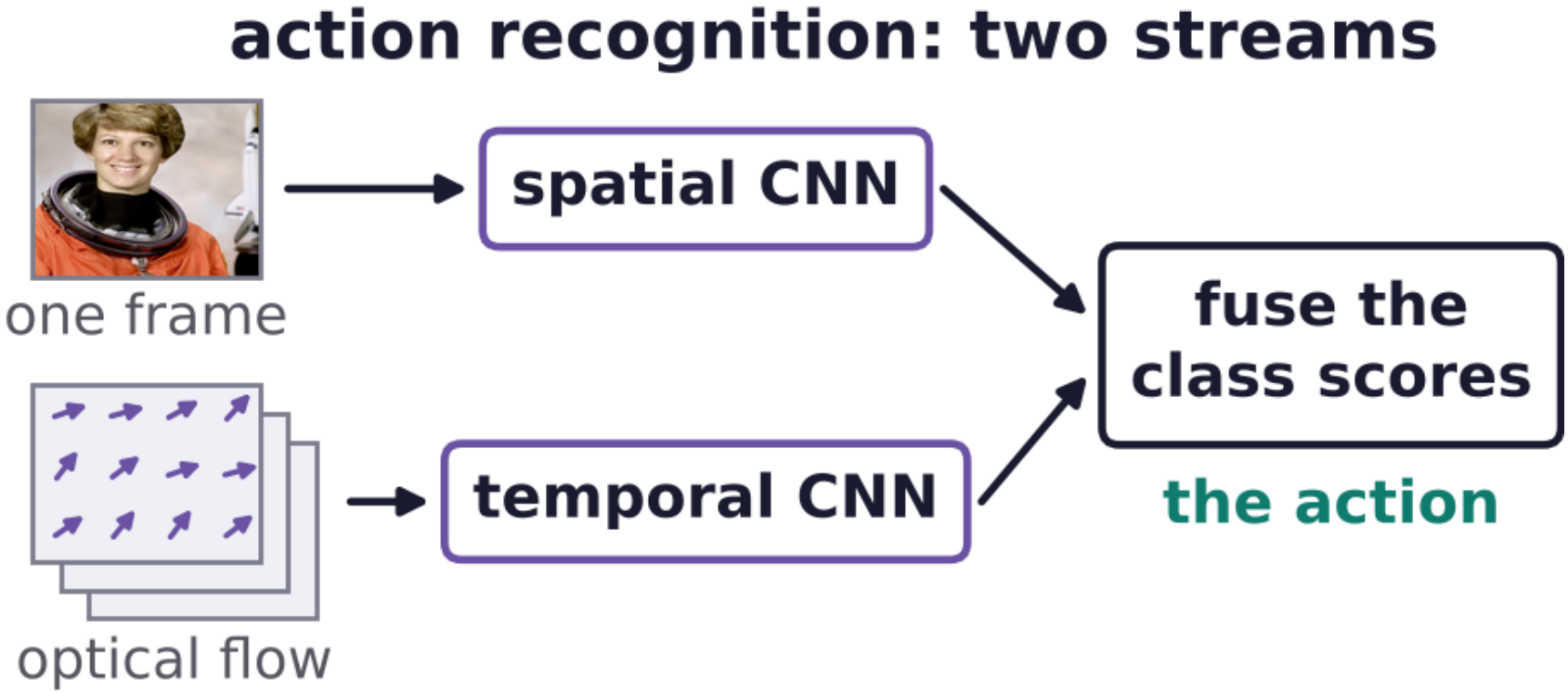


Figure 9: The two-stream network for action recognition, as a schematic with illustrative optical flow.

Image captioning combines vision and language

Image captioning generates a sentence that describes an image. Karpathy and Fei-Fei (2015) combine vision and language features. A CNN computes features of the image and its regions. A recurrent network, which reads or writes a sentence word by word and keeps a hidden state that summarizes the words so far, computes features of the words, which are matched to the image regions. A second recurrent network, given the image features, generates the caption one word at a time, each word chosen from the image and the words before it.

Image captioning combines vision and language

The CNN (purple) turns the photograph into image features. The language network (purple), the second recurrent network, receives them and emits the caption one word at a time (teal), each word chosen from the image features and the words before it.

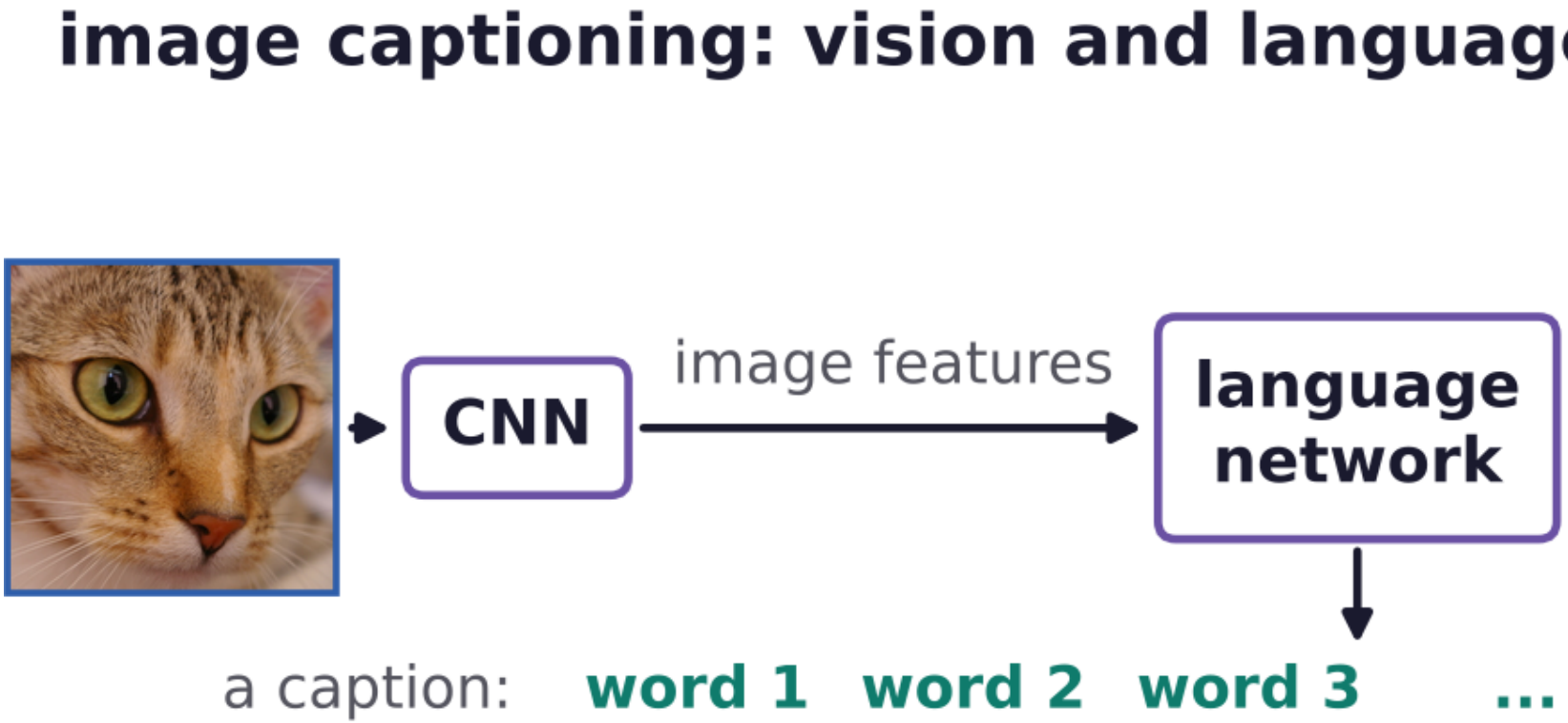


Figure 10: Captioning as a schematic: CNN features of the image condition a language network that emits a caption word by word (the words are placeholders).

Transformers model long-range context adaptively

Transformers were developed for natural language (Vaswani et al., 2017) and adapted to vision as the vision transformer (ViT; Dosovitskiy et al., 2021), which cuts an image into patches and treats them as words. Their ability to model long-range context adaptively has been critical to the success of generative pre-trained transformers (GPTs). It comes from self-attention, among the features of one input, and cross-attention, from one input to another (Lecture 14). A self-attention layer updates the feature vector at each position o with a weighted average over all positions p,

$$y_o = \sum_p \text{softmax}_p(q_o^T k_p)\, v_p,$$

where the query q_o, the key k_p and the value v_p are learned linear projections of the features, and the softmax over p makes the weights positive and sum to 1 (Vaswani et al. also divide $q_o^T k_p$ by $\sqrt{d}$, with d the dimension of the keys). The weights depend on the image itself, so which neurons interact is decided anew for every input (dynamic binding), and any two positions can interact however far apart they are.

Transformers model long-range context adaptively

A neuron of a CNN has a fixed receptive field, with the same weights for every image: the context it uses is fixed in extent and the same for every input. Attention gives transformers some ability to perform sideways interactions, between neurons of the same layer, characteristic of Markov random fields (MRFs; Lecture 6), in which neighboring pixels influence each other's labels.

Transformers have other attractive properties. Trained only to classify, a vision transformer partly segments the object: its attention concentrates on the object (Lecture 13, slide 21). Transformers also share some of the attractive properties of grammatical approaches, which model image patterns as compositions of basic elements (Mumford and Desolneux, 2010, Chapter 3), while being simpler to implement. They have increasingly replaced CNNs for most vision tasks, sometimes in hybrid architectures that combine attention with convolutional filters.

Transformers model long-range context adaptively

In the figure the features are the mean colors of patches of 15×15 pixels, and the weight of a patch is the softmax of ten times its color similarity (a cosine) to the query patch (cyan). The convolution uses the same 3×3 window of patches (white) for every image. A query on the fur attends to similar fur across the face, far outside the window, and a query on the wall attends to the wall.

Transformers model long-range context adaptively

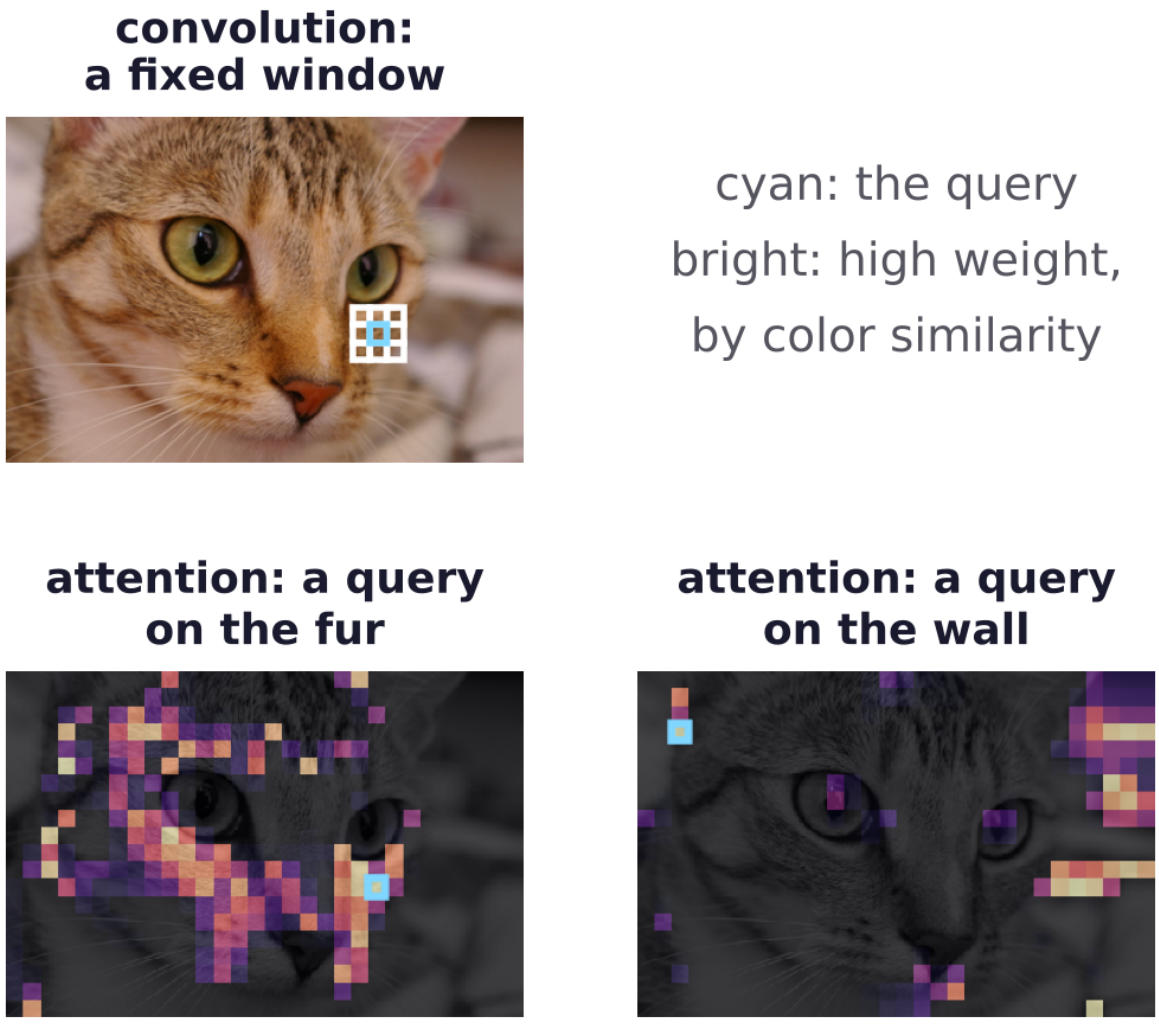


Figure 11: A convolution's fixed window, and the attention weights of two query patches, normalized over all 600 patches and scaled to their maximum (Lecture 14, slide 5).

Networks degrade on another source and are fooled easily

Foundation models are networks trained on enormous amounts of data and then adapted to many tasks. They were developed because a network trained on annotated data from one source degrades when it is tested on data from another source: object detectors trained on ImageNet make mistakes when used by automated cars. Performance on the usual test set is inflated, because the test set comes from the same source as the training set and shares its biases. YK compares this to Ideal Observers (Lecture 2), Bayes-optimal models that can outperform humans in laboratory experiments because they know how the data were generated.

A related weakness: networks can be fooled by minor perturbations of an image, often invisible to humans, called adversarial examples (Szegedy et al., 2014; Lecture 11).

From the BDT perspective, both problems arise because the datasets are not big enough to represent the complexity of natural images: the empirical risk estimates the risk well only for images like those in the dataset.

Networks degrade on another source and are fooled easily

In the upper row AlexNet, trained on ImageNet, labels the photograph and three versions of it; under each image is its most probable label. The grayscale version keeps the label Egyptian cat (teal). The blurred version becomes a puffer fish and the line drawing an envelope (olive), changes that would not fool a human.

In the lower row the perturbation was found by projected gradient descent (PGD; Lecture 11): 20 small steps, each pushing the image toward the class Chihuahua, with every pixel kept within 2 gray levels (out of 255) of the original. The perturbation is shown magnified 40 times. The perturbed image looks identical to the original, and AlexNet labels it Chihuahua.

Networks degrade on another source and are fooled easily

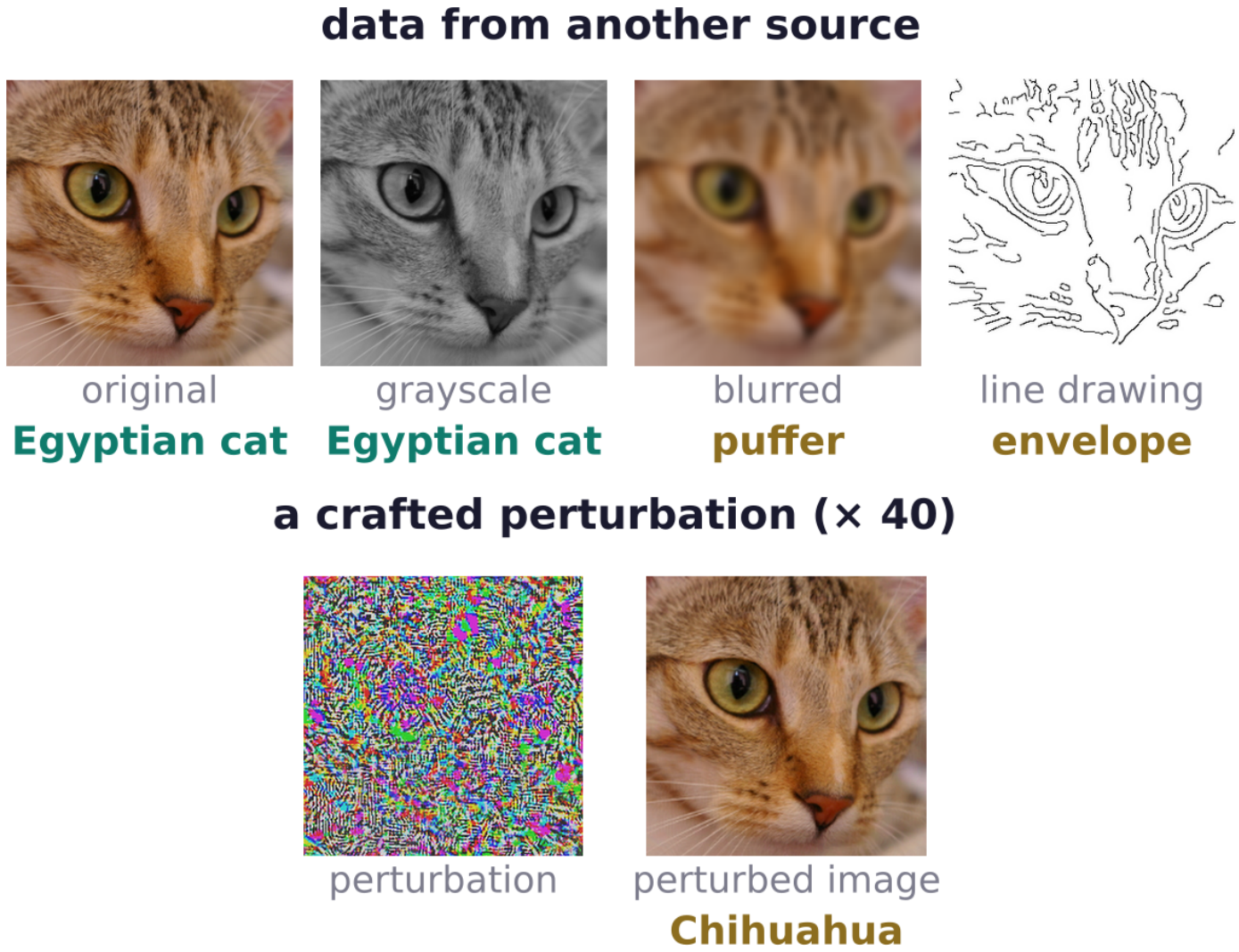


Figure 12: AlexNet on data from another source (Lecture 10, slide 9) and on a crafted perturbation (Lecture 11, slide 8).

Unsupervised learning gives one backbone for many tasks

Foundation models address these problems with several related strategies. The first is unsupervised learning (Lecture 13): network features are learned from enormous numbers of unannotated images, with tasks whose answers come from the images themselves (self-supervised learning). Particularly successful methods are iBOT (Zhou et al., 2022) and DINOv2 (Oquab et al., 2024), self-supervised vision transformers. They combine masked image modeling (MIM), which predicts the features of hidden patches from the visible ones, with self-distillation: a student network learns to match the output of a teacher network, a slowly updated average of the student, on different random crops of the same image. DINOv2 was pre-trained on 142 million curated images, with no labels or text.

Learned from far more varied images than any annotated dataset, the features form a backbone shared by many visual tasks. Each task trains only a small decision head on top of it, with a few annotated examples.

Unsupervised learning gives one backbone for many tasks

Unannotated images train the backbone once. In the figure, a head for the proxy task, whose answers come from the images (orange), serves the unsupervised pre-training; the heads for classification and segmentation (blue) are trained afterward.

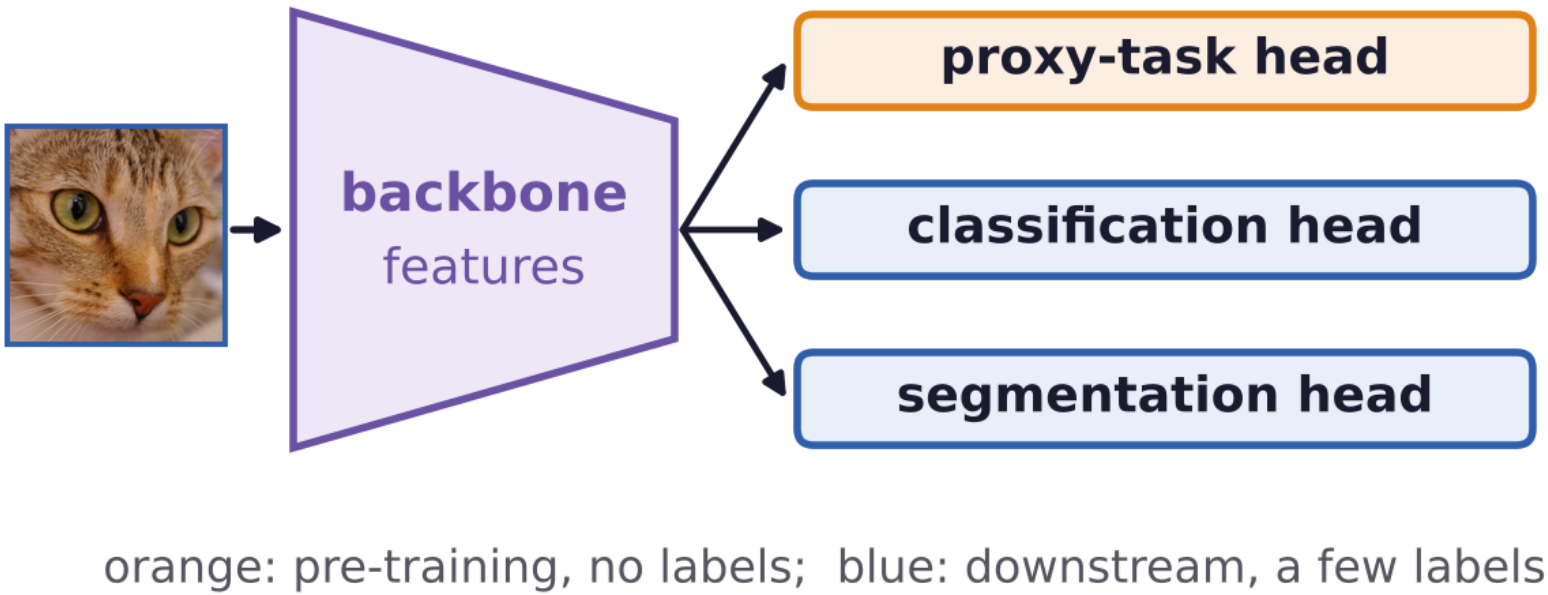


Figure 13: One backbone, several heads (after Lecture 13, slide 11).

Data engines and simulation create enormous datasets

The second strategy creates enormous annotated datasets by model-assisted annotation, which YK calls active learning. A network trained on annotated data predicts labels, called pseudo-labels, for unannotated images; human annotators check them and correct them where necessary; the corrected data retrain the network, which then labels more images.

Segment Anything (SAM; Kirillov et al., 2023) segments the objects in an image. It learns to exploit two cues: the texture within an object typically differs from the texture of the background, and the intensity often changes abruptly at the boundary. A mask marks the pixels of one object. SAM's data engine, the loop of predicting, correcting and retraining, had three stages, from annotators drawing masks with the model's help to fully automatic masks, and produced the dataset SA-1B: 11 million images with 1.1 billion masks.

Data engines and simulation create enormous datasets

The third strategy trains networks on simulated data, like the images of video games, which can be produced in enormous amounts with exact annotation. Depth Anything V2 (Yang et al., 2024) estimates the depth of every pixel from a single image. A teacher network is trained only on about 595 thousand synthetic images, whose depth is known exactly from the rendering. The teacher then labels about 62 million real images with pseudo-depths, and student networks are trained on these pseudo-labels alone.

These foundation models suggest that, for some visual tasks, there is a critical dataset size beyond which the algorithms work on almost all real-world images. From the BDT perspective, this suggests that there is no need for priors and likelihoods, because networks can be trained to model the posterior $P(W \mid I)$ directly.

In the figure each half follows its arrows. Purple boxes are networks; orange marks labels, made by the annotators or by the teacher, and the resulting dataset; the blue box holds the rendered images.

Data engines and simulation create enormous datasets

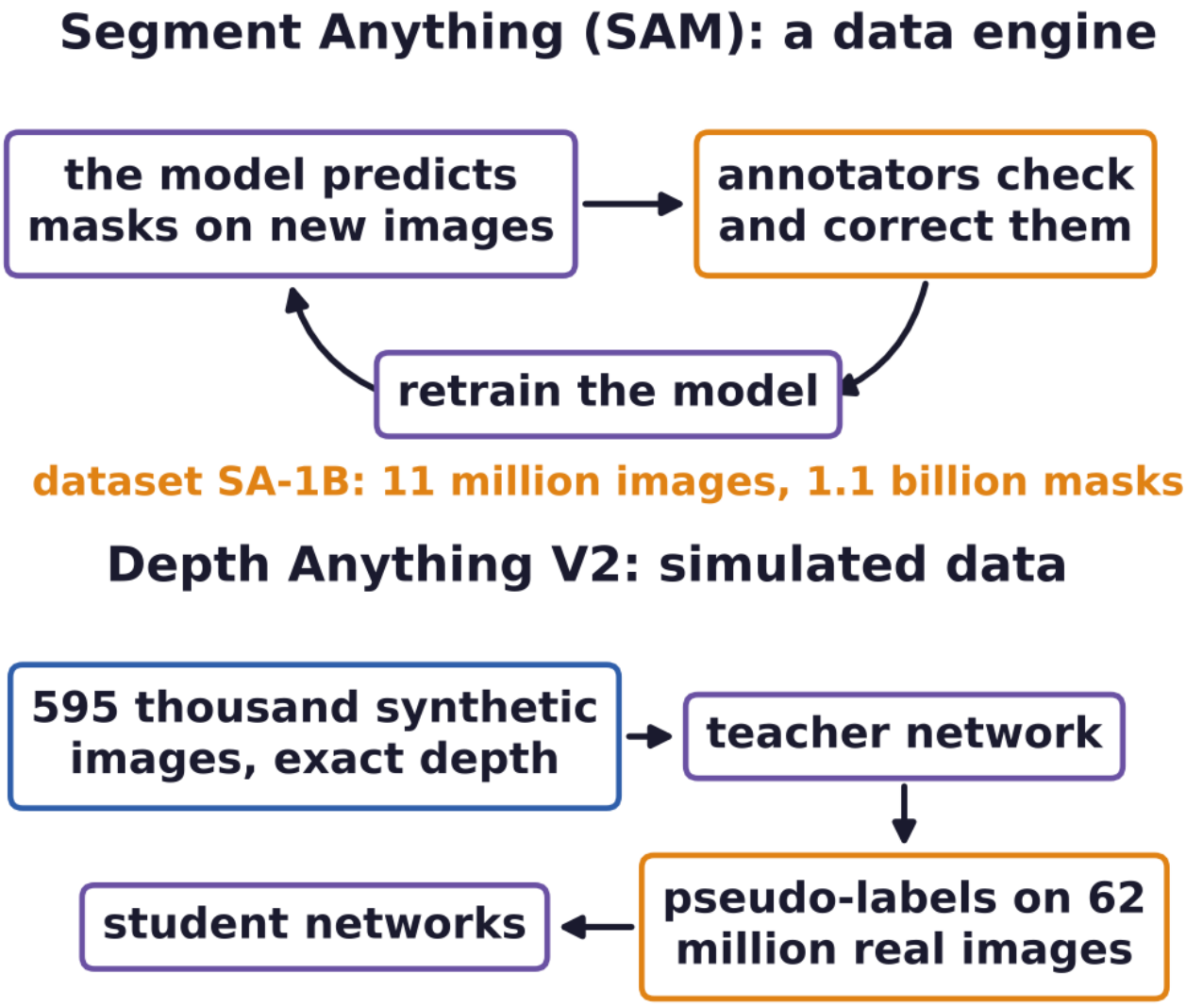


Figure 14: Segment Anything's data engine and the teacher and students of Depth Anything V2, drawn after their descriptions in the papers.

Vision language models exploit common-sense knowledge

The success of captioning encouraged models that let vision use large language models (LLMs). An LLM is a transformer trained on very large amounts of text, and such models capture common-sense knowledge about the world (Devlin et al., 2019).

Contrastive language-image pre-training (CLIP; Radford et al., 2021) trains an image encoder and a text encoder, networks that map an image or a caption to a vector, on 400 million image-caption pairs. In a batch of N pairs, the encodings of each image and its own caption are made similar, and those of the $N^2 - N$ mismatched combinations dissimilar, so images and texts are mapped into one shared space (Lecture 13). A vision language model (VLM) maps the features of an image into the input of an LLM, as if they were text features, and the combined transformer is trained on paired images and texts.

VLMs perform amazingly well on many tasks, such as visual question answering (VQA): answering a question about an image in words. They are, however, black boxes with uninterpretable failure modes, and they hallucinate, stating things the image does not show.

Vision language models exploit common-sense knowledge

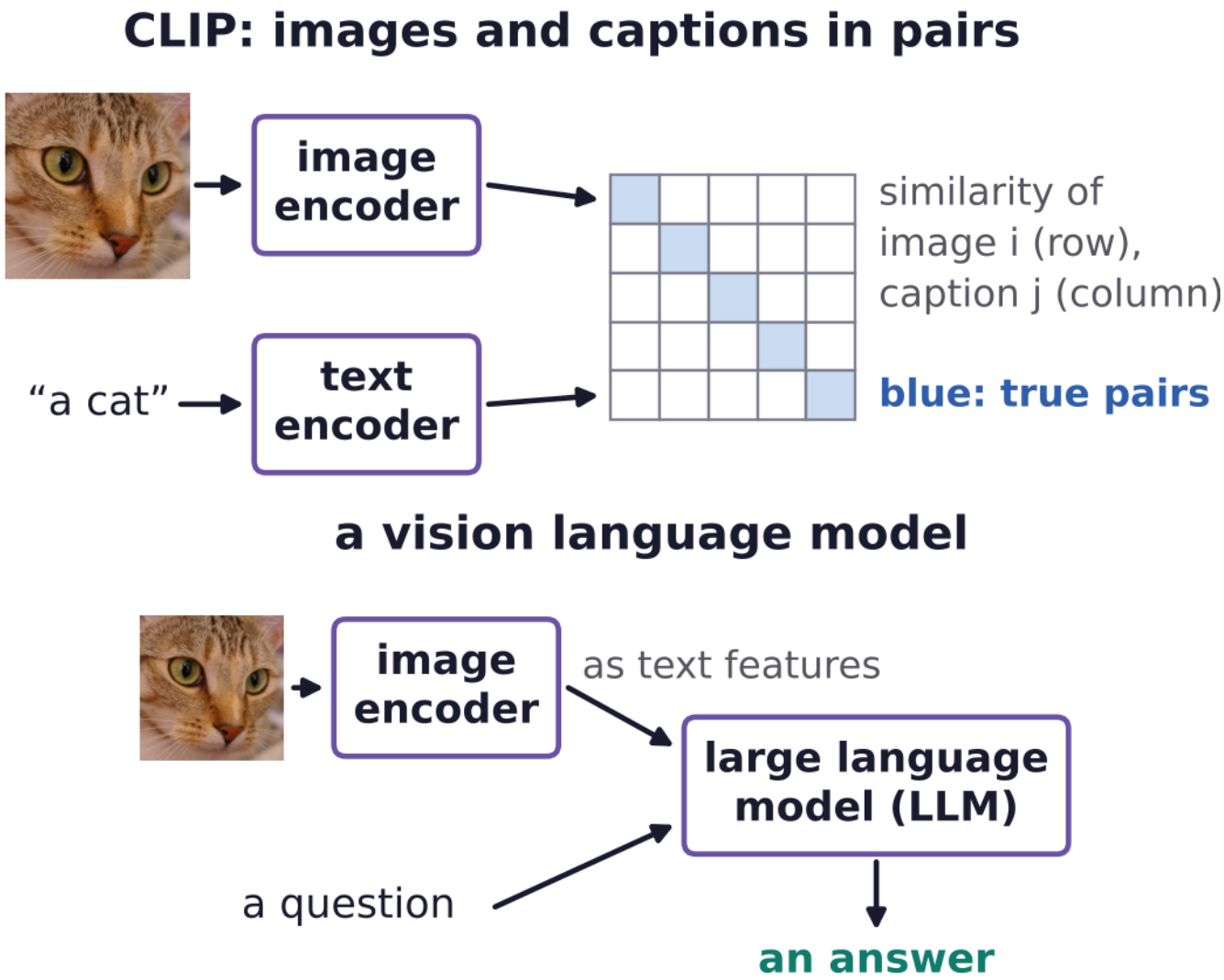


Figure 15: CLIP (rows: images; columns: captions; blue: the true pairs) and a vision language model. The caption "a cat", the question and the answer are placeholders.

GANs train a generator against a discriminator

Generative networks now produce images so realistic that they have won prizes in art competitions. A generative adversarial network (GAN; Goodfellow et al., 2014) trains a generator G, essentially an upside-down CNN, which maps a random code z, a vector of random numbers, to an image $G(z)$, from coarse to fine. It competes with a discriminator D, a CNN trained to tell real images from synthetic ones: $D(I)$ is the probability it gives to I being real. The two are trained on

$$\min_G \max_D \; \mathbb{E}_I \big[\log D(I) \big] + \mathbb{E}_z \big[\log \big(1 - D(G(z)) \big) \big],$$

where the first expectation is over real images and the second over random codes. D maximizes the objective by giving real images probabilities near 1 and synthetic ones near 0; G minimizes it by making $D(G(z))$ large, so that its images become hard to tell from real ones. At the optimum the distribution of $G(z)$ equals the distribution of real images (Goodfellow et al., 2014). G is then a generative model: images can be sampled from it, although the probability of a given image cannot be evaluated.

GANs train a generator against a discriminator

The random code z (the purple column) is mapped by G to a synthetic image $G(z)$, drawn as a gray placeholder. D receives synthetic images and real ones (the photograph) and outputs the probability that each is real.

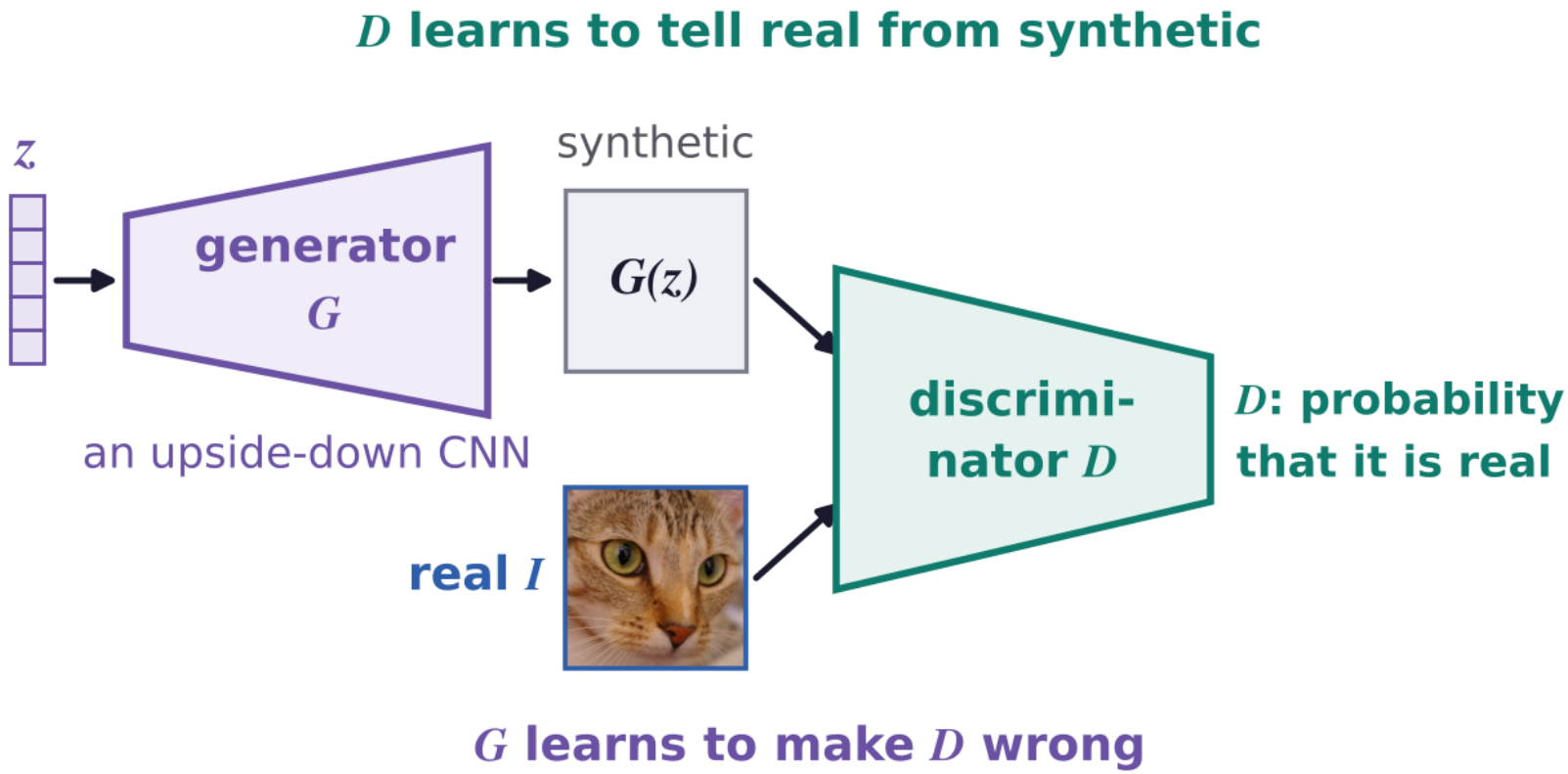


Figure 16: A generative adversarial network, as a schematic.

Diffusion models learn to reconstruct images from noise

The dominant current approaches are diffusion models, trained like an auto-encoder, which reconstructs its input from a hidden (latent) code: an image is diffused into noise, and a network is trained to reconstruct it. In the formulation of Ho et al. (2020), each of 1000 steps shrinks the image slightly and adds a little Gaussian noise; after t steps the image I_0 has become

$$I_t = \sqrt{\gamma_t}\, I_0 + \sqrt{1-\gamma_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, \mathrm{Id}), \qquad \gamma_t = \prod_{s=1}^{t} (1-\beta_s),$$

where the β_s are small noise variances; γ_t falls from 1 to almost 0, so I_{1000} is nearly pure noise. A network ϵ_ω, with weights ω, learns to predict the noise from I_t and t by minimizing $\mathbb{E}\,|\epsilon - \epsilon_\omega(I_t, t)|^2$ over ω, the expectation being over images, steps and noise. Generation starts from noise and removes the predicted noise step by step. A text caption can prime the generation, entering the network by the attention of transformers (Saharia et al., 2022).

Diffusion models learn to reconstruct images from noise

The figure uses the schedule of Ho et al.: β rises linearly from 10^{-4} to 0.02, and γ_t is 0.90 at $t = 100$, 0.52 at $t = 250$, 0.079 at $t = 500$ and 4×10^{-5} at $t = 1000$. These models produce enormously varied, detailed images, and they are very exciting from the Bayesian perspective: with the computer graphics of game engines, they suggest that it is becoming practical to develop the distributions that generate even the most complex images, the likelihood $P(I \mid W)$ that Bayes requires.

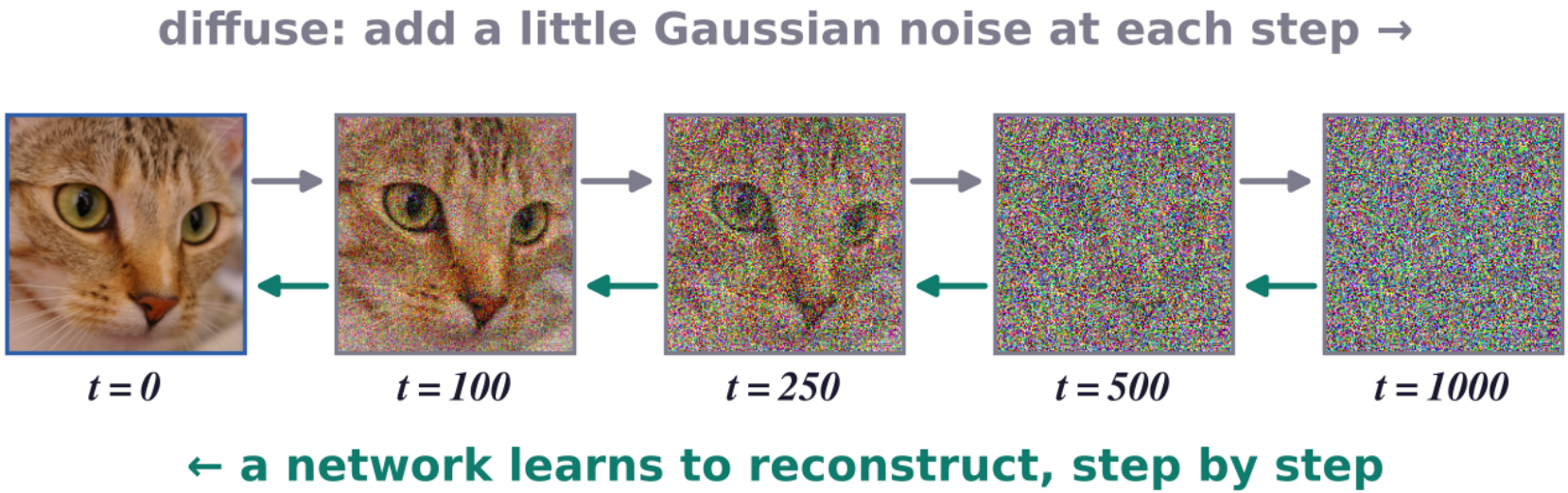


Figure 17: The photograph diffused with the schedule of Ho et al. (one draw of the noise); a network learns the reverse steps.

We conjecture that Bayes is needed for responsible AI

Computer vision is a very dynamic field whose scope is expanding to include language, reasoning and much of artificial intelligence (AI) and high-level cognition, and it is a major industry. As with AI in general, there are concerns that it relies too much on black-box techniques, which are complex and hard to interpret. Responsible AI should have understandable failure modes and performance guarantees, and be controllable.

BDT contrasts heavily data-driven feedforward networks with the Bayesian alternative. YK conjectures that Bayes, though technically more challenging, will be necessary to achieve responsible AI. Bayes enables a duality between analysis and synthesis. Analysis estimates the world state W from the image I; synthesis generates the image that W would produce, through $P(I \mid W)$; comparing the synthesized image with the input checks the interpretation and corrects it (analysis by synthesis). Bayes also uses structured representations: 3D world models of objects, agents, their interactions, and knowledge about the world. This fits theories in which children are "baby scientists", who learn to model the world by making predictions and testing them by experiments.

We conjecture that Bayes is needed for responsible AI

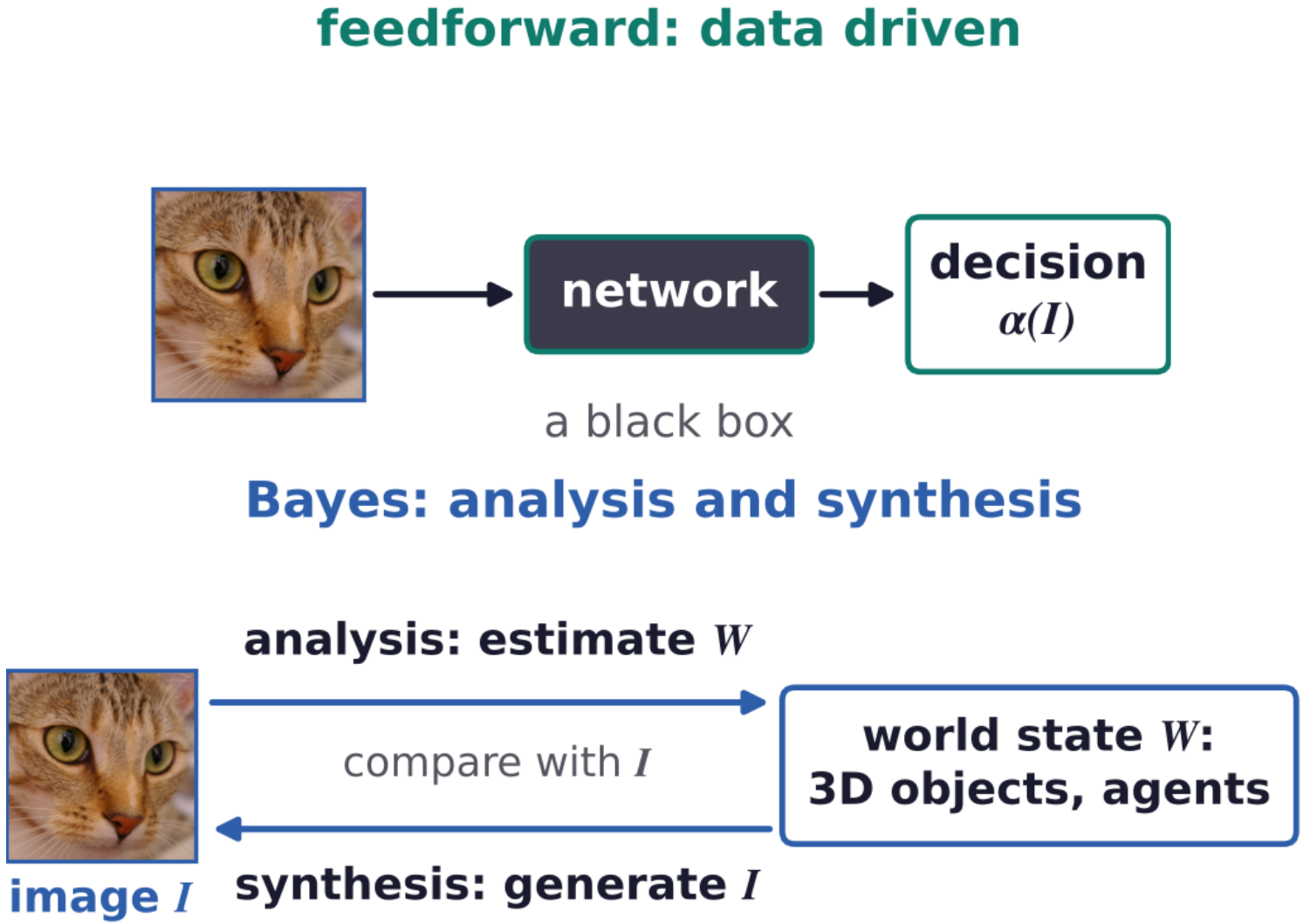


Figure 18: A feedforward network (teal) maps the image to a decision as a black box. Bayes (blue): analysis estimates the world state W from I, synthesis generates an image from W, and the two are compared.

Computer vision may increasingly shape cognitive science

The second conjecture of YK is that computer vision will increasingly influence the cognitive science of vision. Computer vision provides a rich set of tools for constructing computational models of vision that work on realistic images, and these models can be tested by behavioral experiments. Ideal Observer studies will give insight into human visual abilities, which may in turn improve computer vision.

Computer vision and AI techniques are already used to predict brain activity measured by functional magnetic resonance imaging (fMRI) and by electroencephalography (EEG), and to understand the functions of the visual areas and the relationships between them. Feedforward networks, originally inspired by neuroscience, increasingly predict multi-electrode recordings of neurons (Kell and McDermott, 2019) and address the neural basis of cognition. This may give better scientific understanding, and also novel neural architectures for AI.

Computer vision may increasingly shape cognitive science

Each double arrow joins the models to one kind of evidence. The orange boxes are data; the blue box is Ideal Observers, Bayesian models of the same tasks. The models are tested against each, and the comparison feeds back into them.

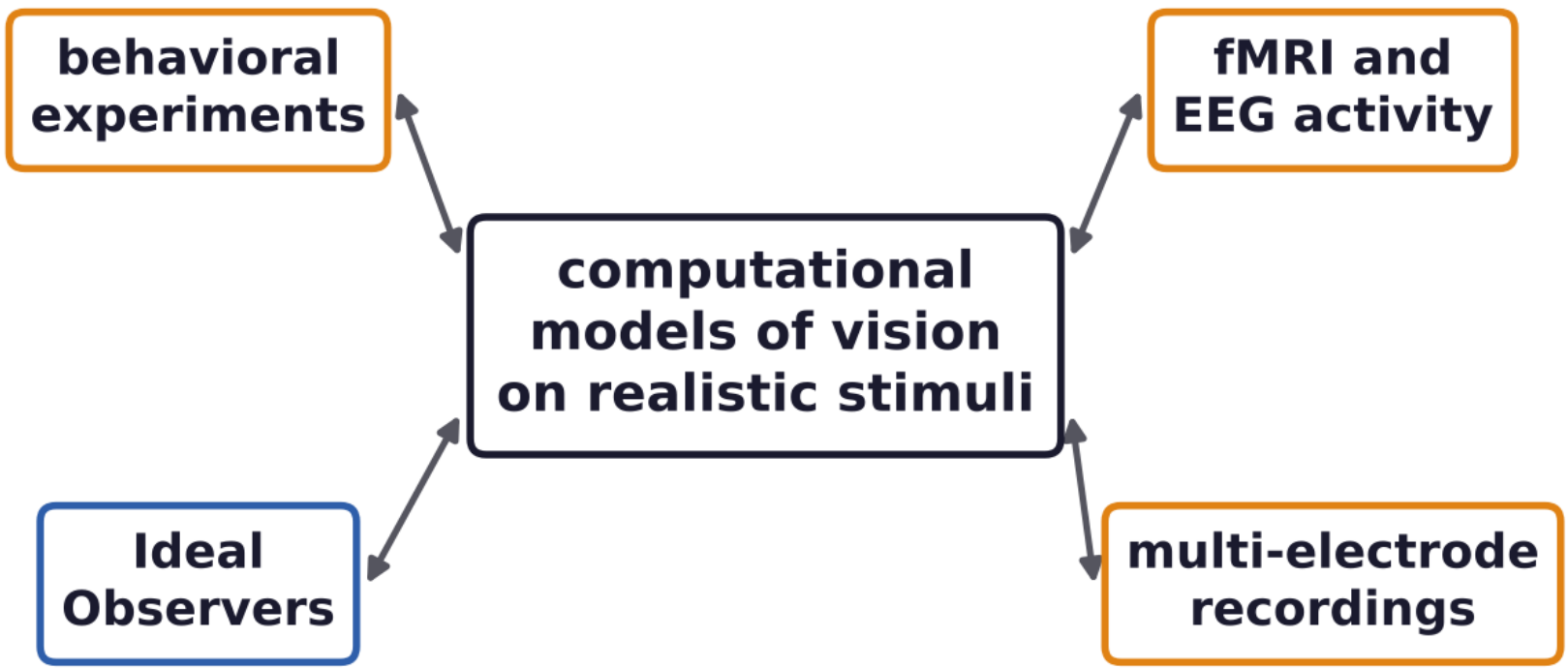


Figure 19: Computational models of vision on realistic stimuli, tested against three kinds of data and against Ideal Observers.

Summary

- BDT: learn $P(I \mid W)$ and $P(W)$ (may need less data), or learn $P(W \mid I)$ and the rule directly (big datasets).
- Features: hand-designed filters, then hierarchical networks inspired by the ventral stream.
- A network is a class of decision rules; backpropagation and SGD minimize the empirical risk.
- CNNs succeeded with large annotated datasets and GPUs, then spread to every annotated visual task.
- Transformers add long-range, adaptive context; foundation models address datasets that were too small.
- At a critical dataset size the posterior may be learned directly; VLMs add knowledge from language.
- Generative models (GANs, diffusion) suggest that $P(I \mid W)$ is practical even for complex images.
- Conjectures: Bayes is needed for responsible AI; computer vision will shape cognitive science.

References

[1] A. Yuille and D. Kersten. Computer vision and its relationship to cognitive science: a perspective from Bayes decision theory. Manuscript (YK), 2025.

[2] A. L. Yuille. Probabilistic Models of the Visual Cortex, slides of Lectures 2, 10, 11, 13 and 14. Johns Hopkins University, 2026.

[3] J. O. Berger. *Statistical Decision Theory and Bayesian Analysis*, 2nd ed. Springer, 1985.

[4] T. L. Griffiths, N. Chater and J. B. Tenenbaum. *Bayesian Models of Cognition: Reverse Engineering the Mind*. MIT Press, 2024.

[5] S. Konishi, A. L. Yuille, J. M. Coughlan and S. C. Zhu. Statistical edge detection: learning and evaluating edge cues. *IEEE TPAMI*, 25(1):57–74, 2003.

[6] K. Fukushima. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics*, 36(4):193–202, 1980.

[7] M. Riesenhuber and T. Poggio. Hierarchical models of object recognition in cortex. *Nature Neuroscience*, 2(11):1019–1025, 1999.

References

[8] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.

[9] P. Felzenszwalb, D. McAllester and D. Ramanan. A discriminatively trained, multiscale, deformable part model. *CVPR*, 2008.

[10] A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet classification with deep convolutional neural networks. *NIPS*, 1097–1105, 2012.

[11] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh et al. ImageNet large scale visual recognition challenge. *IJCV*, 115(3):211–252, 2015.

[12] K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. *CVPR*, 770–778, 2016.

[13] J. Long, E. Shelhamer and T. Darrell. Fully convolutional networks for semantic segmentation. *CVPR*, 3431–3440, 2015.

References

[14] S. Xie and Z. Tu. Holistically-nested edge detection. *ICCV*, 1395–1403, 2015.

[15] X. Chen and A. Yuille. Articulated pose estimation by a graphical model with image dependent pairwise relations. *NIPS*, 1736–1744, 2014.

[16] K. Simonyan and A. Zisserman. Two-stream convolutional networks for action recognition in videos. *NIPS*, 568–576, 2014.

[17] A. Karpathy and L. Fei-Fei. Deep visual-semantic alignments for generating image descriptions. *CVPR*, 3128–3137, 2015.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin. Attention is all you need. *NIPS*, 5998–6008, 2017.

[19] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai et al. An image is worth 16x16 words: transformers for image recognition at scale. *ICLR*, 2021.

References

[20] D. Mumford and A. Desolneux. *Pattern Theory: The Stochastic Analysis of Real-World Signals*. A K Peters, 2010.

[21] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow and R. Fergus. Intriguing properties of neural networks. *ICLR*, 2014.

[22] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. Yuille and T. Kong. iBOT: image BERT pre-training with online tokenizer. *ICLR*, 2022.

[23] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec et al. DINOv2: learning robust visual features without supervision. *TMLR*, 2024.

[24] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland et al. Segment anything. *ICCV*, 4015–4026, 2023.

[25] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng and H. Zhao. Depth Anything V2. *NeurIPS*, 2024.

References

[26] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 4171–4186, 2019.

[27] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh et al. Learning transferable visual models from natural language supervision. *ICML*, 8748–8763, 2021.

[28] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville and Y. Bengio. Generative adversarial nets. *NIPS*, 2672–2680, 2014.

[29] J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. *NeurIPS*, 6840–6851, 2020.

[30] C. Saharia, W. Chan, S. Saxena, L. Li, J. Whang et al. Photorealistic text-to-image diffusion models with deep language understanding. *NeurIPS*, 36479–36494, 2022.

[31] A. J. E. Kell and J. H. McDermott. Deep neural network models of sensory systems: windows onto the role of task constraints. *Current Opinion in Neurobiology*, 55:121–132, 2019.